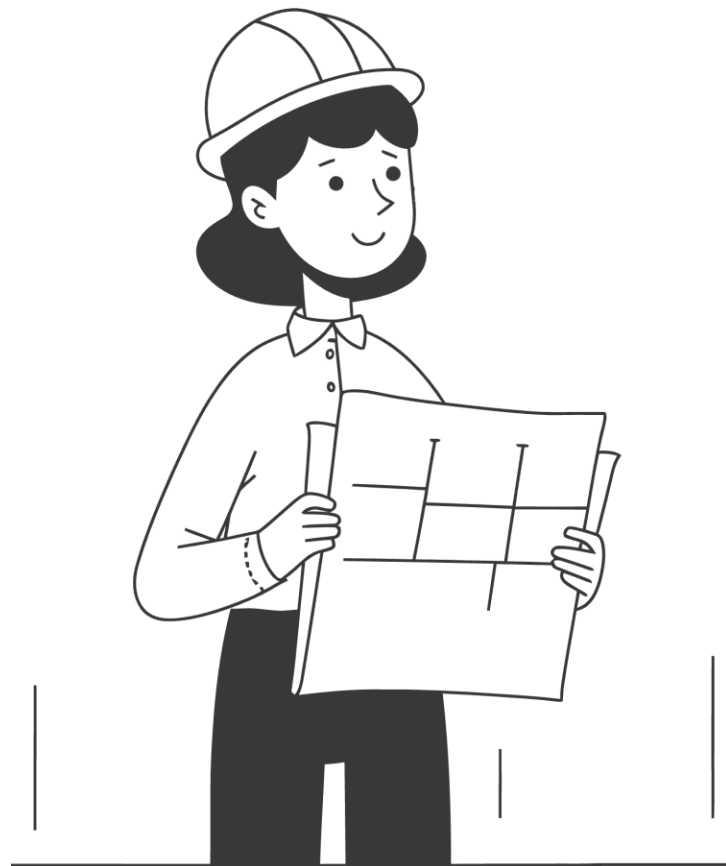


Page Object Model

Session 5 — Playwright Automation Bootcamp

QA ENGINEERS · SDETS · AUTOMATION ENGINEERS



Why Page Object Model?

Change one place, not twenty.

✗ Without POM — duplicated everywhere

```
// test 1, test 2, test 3, test 4...  
await page.getByTestId('login-email').fill('user@example.com')  
await page.getByTestId('login-password').fill('pass')  
await page.getByTestId('login-submit').click()
```

✓ With POM — one method call

```
await loginPage.login('user@example.com', 'pass')
```

If login-submit is renamed: fix LoginPage.login() once. All tests pass.

Three Rules of POM

1. Page objects describe **HOW** to interact — specs describe **WHAT** to test
2. Locators belong in page objects, not in specs
3. Page object methods should assert their own outcome (leave the page in a known state)

POM Folder Structure

Where Does Everything Go? Each file has one job.

pages

HOW to interact with each page

utils

Data generation and shared helpers



fixtures

HOW to inject page objects into tests

specs

WHAT to test — no locators or UI actions

pages/

HOW to interact with each page

fixtures/

HOW to inject page objects into tests

utils/

Data generation and shared helpers

specs/

WHAT to test — no locators, no UI interactions

```
tests/
├── pages/
│   ├── base.page.ts      shared: navigate(), waitForLoad()
│   ├── login.page.ts     login(), assertLoginError()
│   ├── products.page.ts  search(), filterByCategory(), assertResultCount()
│   └── cart.page.ts      addItem(), removeItem(), assertTotal()
├── fixtures/
│   └── test-fixtures.ts  extends test with loginPage, productsPage
├── utils/
│   └── data-builder.ts   buildUser(), buildProduct()
└── specs/
    ├── login.spec.ts
    └── products.spec.ts
```

BasePage & LoginPage

Inheritance: BasePage gives every page a shared foundation.

BasePage — shared behaviour

```
export class BasePage {  
  constructor(readonly page: Page) {}  
  async navigate(path: string): Promise<void> {  
    await this.page.goto(path)  
  }  
}
```

readonly page

Prevents accidental reassignment

LoginPage — extends BasePage

```
export class LoginPage extends BasePage {  
  async login(email: string, password: string) {  
    await this.page.getByTestId('login-email').fill(email)  
    await this.page.getByTestId('login-password').fill(password)  
    await this.page.getByTestId('login-submit').click()  
    await expect(this.page.getByTestId('nav-logout')).toBeVisible()  
  }  
  async assertLoginError(message: string) {  
    await expect(this.page.getByTestId('login-error')).toHaveText(message)  
  }  
}
```

Self-asserting login()

Leaves the page in a verified state

Lifecycle Hooks

Eliminate setup duplication — navigate once, reused by all tests.

✗ Without hooks

```
test('search', async ({ page }) => {  
  await page.goto('/products') // duplicated  
  ...  
})  
test('filter', async ({ page }) => {  
  await page.goto('/products') // duplicated again  
  ...  
})
```

✓ With beforeEach

```
test.describe('Products page', () => {  
  test.beforeEach(async ({ page }) => {  
    await page.goto('/products')  
  })  
  test('search', async ({ page }) => { ... })  
  test('filter', async ({ page }) => { ... })  
})
```

1

beforeAll

Once before all tests — no `page` fixture

2

beforeEach

Before every test — receives `page`, `context`

3

afterEach

After every test — receives `testInfo.status`

4

afterAll

Once after all tests — no `page` fixture

⚠ **Most common mistake:** Using `page` inside `beforeAll` — it does not work. `page` is per-test only.

Custom Fixtures

Ask for a page object, get one. No more `new LoginPage(page)` boilerplate in every test.

tests/fixtures/test-fixtures.ts

```
type Fixtures = { loginPage: LoginPage; productsPage: ProductsPage }

export const test = base.extend<Fixtures>({
  loginPage: async ({ page }, use) => { await use(new LoginPage(page)) },
  productsPage: async ({ page }, use) => { await use(new ProductsPage(page)) },
})
```

Tests now receive page objects directly

```
import { test, expect } from '../fixtures/test-fixtures'

test('search after login', async ({ loginPage, productsPage }) => {
  await loginPage.navigate('/login')
  await loginPage.login('standard_user@example.com', 'Password123!')
  await productsPage.search('Keyboard')
  await productsPage.assertResultCount(1)
})
```

✓ **The test body:** Every line is a business action. No locators. No raw page calls. A business analyst could read this and understand it.

Data Builders

Never hard-code test data. Hard-coded names cause unique constraint errors in parallel runs.

Factory function with defaults & overrides

```
export function buildProduct(overrides?: Partial<{
  name: string; price: number;
  category: string; stock: number;
}>) {
  return {
    name: 'Product-' + Date.now(), // unique per
    run
    description: 'Test product',
    price: 9.99,
    category: 'Electronics',
    stock: 10,
    ...overrides,
  }
}
```

Usage patterns

```
// unique defaults
const product = buildProduct()

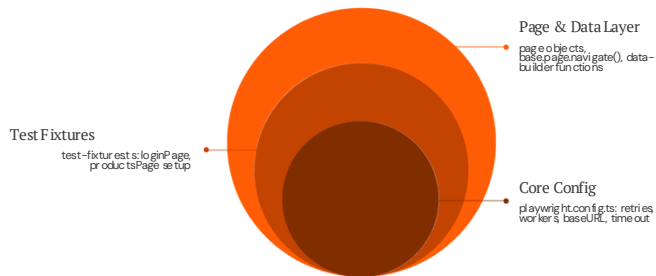
// one field overridden
const expensive = buildProduct({ price: 999.99 })

// multiple overrides
const software = buildProduct({
  name: 'My Course',
  category: 'Software'
})
```

 This is the `Partial<T>` pattern from Session 1 TypeScript, applied to real test automation.

Putting It All Together

How all the pieces connect into one complete, readable test.



```
test('created product appears in search', async ({ productsPage, request }) => {  
  const product = buildProduct({ name: 'Lab Test Widget' })  
  await request.post('/api/products', { data: product }) // API precondition  
  await productsPage.navigate('/products')  
  await productsPage.search('Lab Test Widget')  
  await productsPage.assertResultCount(1)  
})
```

Session 1

TypeScript interfaces

Session 2

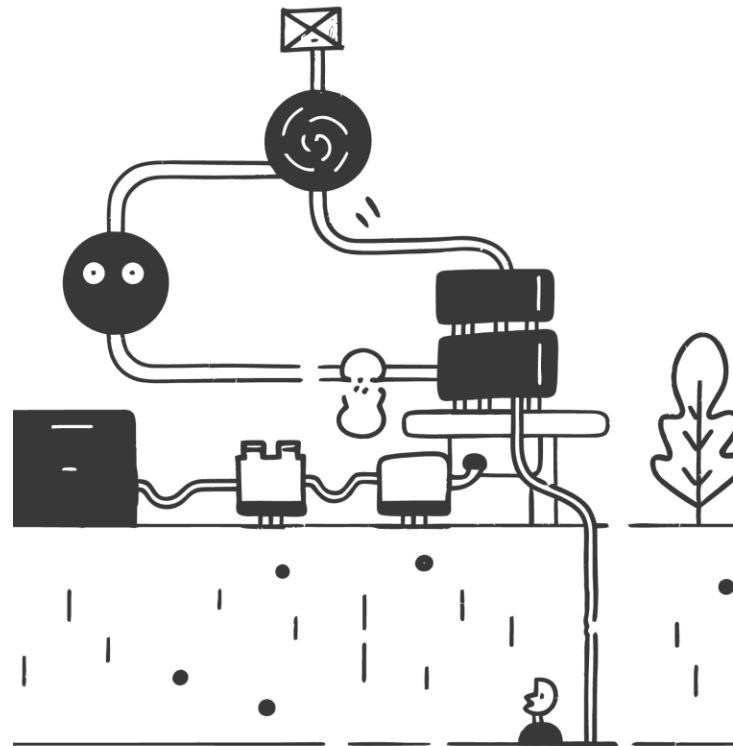
Locators & auto-wait

Session 3

PCM & fixtures

Session 4

API preconditioning



Hooks + POM Integration

The standard pattern for authenticated tests — login once in `beforeEach`, tests focus on their own concerns.

❌ Before — login duplicated

```
test('view cart', async ({ page }) => {  
  await new LoginPage(page)  
    .login('user@example.com', 'pass') // duplicated  
  ...  
})
```

✅ After — login in beforeEach

```
test.describe('authenticated flows', () => {  
  test.beforeEach(async ({ loginPage }) => {  
    await loginPage.navigate('/login')  
    await loginPage.login('standard_user@example.com',  
      'Password123!')  
  })  
  test('cart is accessible', async ({ page }) => {  
    await page.goto('/cart')  
    await expect(page.getByTestId('cart-heading')).toBeVisible()  
  })  
  test('orders page loads', async ({ page }) => {  
    await page.goto('/orders')  
    await expect(page.getByTestId('orders-heading')).toBeVisible()  
  })  
})
```

✅ **Bonus:** If login breaks, all tests in the describe fail with the same error — immediately identifies authentication as the root cause.